

# Práctica 2

Trabajando con el Turtlebot 3 en simulación

Abel Gandía Ruiz

<https://github.com/danisty/tar>

# Índice

---

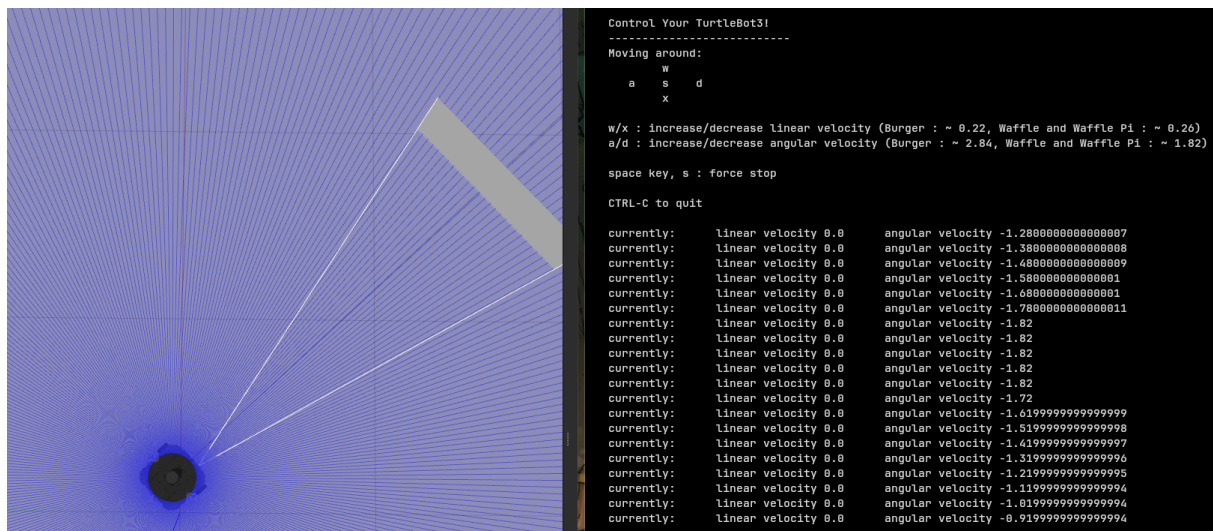
|                                                            |    |
|------------------------------------------------------------|----|
| Parte 1: Primeros Pasos con el Turtlebot y Odometría ..... | 3  |
| Preguntas teóricas .....                                   | 3  |
| Ejercicios .....                                           | 6  |
| Parte 2: Resolución de un Laberinto .....                  | 19 |
| Preguntas teóricas .....                                   | 19 |
| Ejercicios .....                                           | 20 |

# Parte 1: Primeros Pasos con el Turtlebot y Odometría

## Preguntas teóricas

Para esta primera parte se lanza el Turtlebot 3 **Waffle** en **Gazebo** y se teleopera con el paquete **turtlebot3\_teleop**. Esto nos permite estudiar los mecanismos de comunicación que emplea ROS 2 para controlar al robot.

```
export TURTLEBOT3_MODEL=waffle
ros2 run turtlebot3_teleop teleop_keyboard
```



### PREGUNTA 1

¿Cuál es el topic en el cual se debe publicar la información para que el robot se mueva?

El topic sobre el que se publica para mover al robot es **/cmd\_vel**. Se puede comprobar inspeccionando el nodo de teleoperación:

```
ros2 node info /teleop_keyboard
```

```
/teleop_keyboard
Subscribers:

Publishers:
  /cmd_vel: geometry_msgs/msg/Twist
  ...
Service Servers:
  ...
  ...
```

### PREGUNTA 2

## ¿Cuál es el tipo de mensaje que se publica?

El tipo de mensaje publicado en `/cmd_vel` es `geometry_msgs/msg/Twist`. Se puede verificar con:

```
ros2 topic info /cmd_vel
```

```
Type: geometry_msgs/msg/Twist
Publisher count: 1
Subscription count: 1
```

El mensaje `Twist` contiene dos vectores de tres componentes cada uno:

```
ros2 interface show geometry_msgs/msg/Twist
```

```
# This expresses velocity in free space broken into its linear and angular parts.
Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
```

Para el Turtlebot 3, al tratarse de un robot diferencial que se mueve sobre un plano, únicamente se utilizan `linear.x` (velocidad de avance) y `angular.z` (velocidad de giro).

### PREGUNTA 3

¿Qué tipo de movimientos puede realizar el robot? ¿Cuáles son los ejes de movimientos positivos?

El Turtlebot 3 es un robot de **tracción diferencial** con dos ruedas motrices. Esto le permite realizar dos tipos de movimiento:

1. **Movimiento lineal** a lo largo del eje X del robot: `linear.x > 0` avanza hacia adelante, `linear.x < 0` retrocede.
2. **Movimiento rotacional** alrededor del eje Z (vertical): `angular.z > 0` gira en sentido **antihorario** (izquierda), `angular.z < 0` gira en sentido **horario** (derecha).

No puede desplazarse lateralmente (no tiene movimiento en el eje Y), por tanto, si le mandamos un giro de  $-20^\circ$ , el robot girará hacia la **derecha**.

### PREGUNTA 4

¿El robot gira sobre su eje siempre? ¿Qué comando de odometría se le está mandando a los motores para que sea así?

Sí, el Turtlebot 3 gira sobre su propio eje cuando se le envía únicamente velocidad angular (`angular.z != 0`) con velocidad lineal nula (`linear.x = 0`). En este caso, ambas ruedas giran a la misma velocidad pero en **sentidos opuestos**, haciendo que el punto medio del eje de las ruedas se mantenga fijo.

### PREGUNTA 5

¿Cuál es la unidad de magnitud de las velocidades lineales y angulares?

---

Las unidades en ROS 2 siguen el sistema internacional (SI):

- › **Velocidad lineal:** metros por segundo (m/s)
- › **Velocidad angular:** radianes por segundo (rad/s)

## Ejercicios

### EJERCICIO 1

Nodo `movimiento.py` con 4 modos de movimiento y nodo `dibuja_mov.py` para visualización de trayectorias

Se ha creado el paquete `p3_pkg` con las dependencias necesarias (`rclpy`, `geometry_msgs`, `nav_msgs`). Dentro de este paquete se implementan dos nodos: `movimiento.py` para ejecutar las trayectorias y `dibuja_mov.py` para registrarlas y dibujarlas.

Cada modo de movimiento define una cola de pasos `('forward', distancia)` o `('rotate', grados)`. El callback de odometría (`/odom`) compara la posición/ángulo actual con el origen del paso y, cuando se alcanza la tolerancia, pasa al siguiente paso.

El nodo soporta dos modos de control, seleccionables con el flag `--pid`:

- › **Sin PID (por defecto):** velocidad constante; útil para observar el *drift* odométrico en los ejercicios 1-2.
- › **Con PID (`--pid`):** controlador PID sobre la rotación y corrección de *heading* en avance; detallado en el ejercicio 4.

#### MODO 0: MOVIMIENTO LINEAL DE 2 METROS

```
if movement_type == 0:  
    self.steps.append(('forward', 2.0))
```

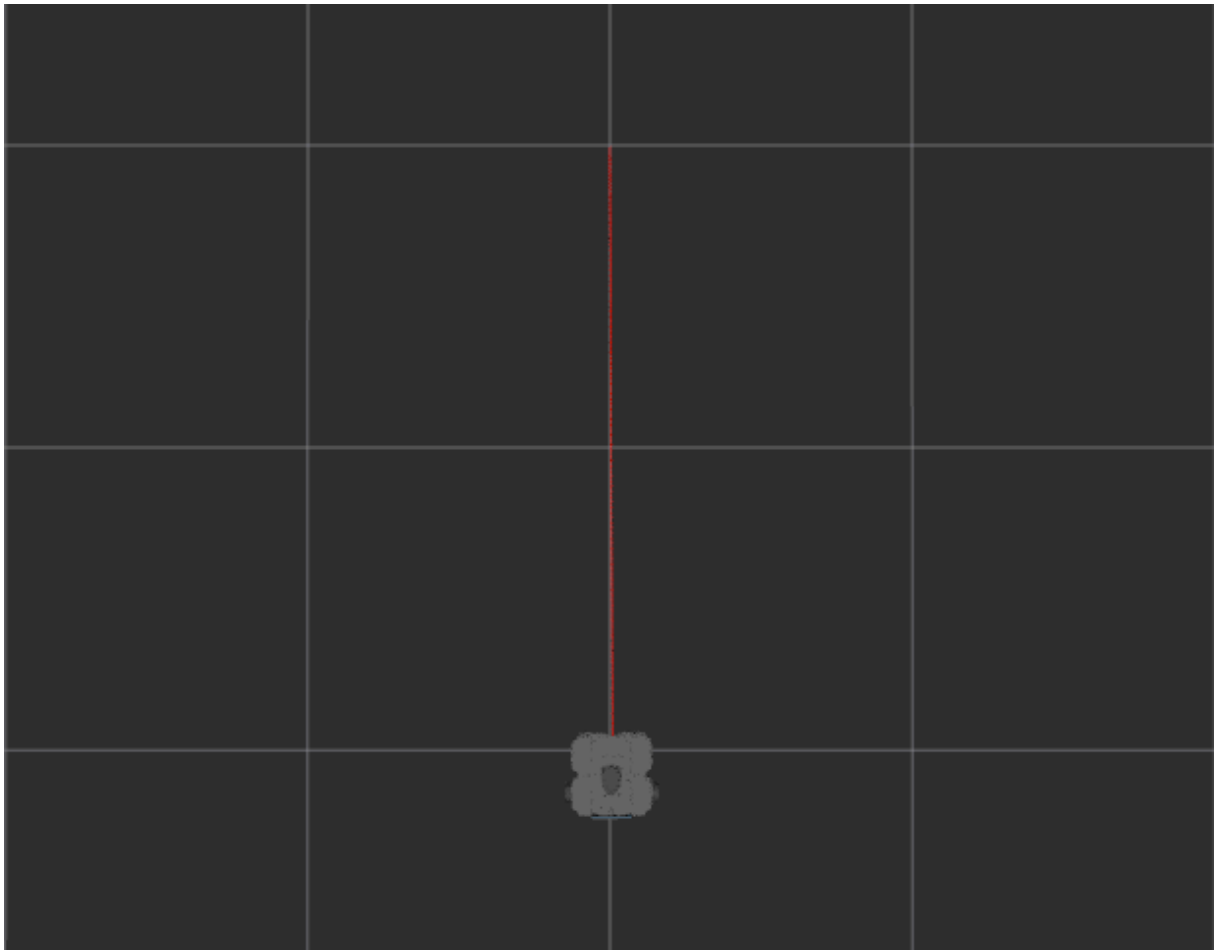
El robot avanza 2 metros en línea recta midiendo la distancia recorrida a través de la odometría.

### PREGUNTA 6

¿Cómo puedo comprobar que el robot haya avanzado esta distancia?

Se puede comprobar suscribiéndose al topic `/odom` de tipo `nav_msgs/msg/Odometry`. Este topic publica la posición estimada del robot. Comparando `pose.pose.position.x` antes y después del movimiento se puede calcular la distancia recorrida con:

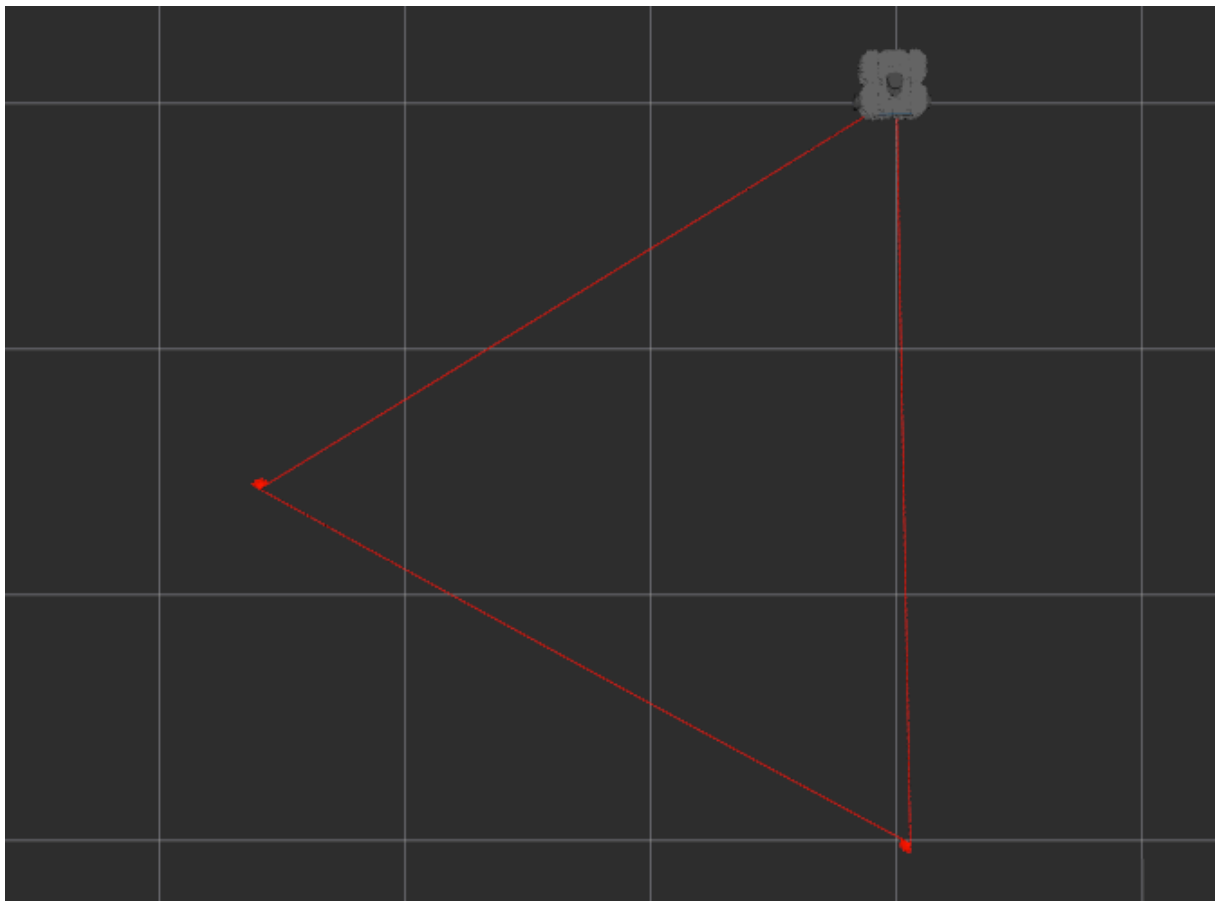
```
ros2 topic echo /odom
```



#### MODO 1: TRIÁNGULO EQUILÁTERO DE 3M DE LADO

```
elif movement_type == 1:  
    for _ in range(3):  
        self.steps.append(('forward', 3))  
        self.steps.append(('rotate', -120))
```

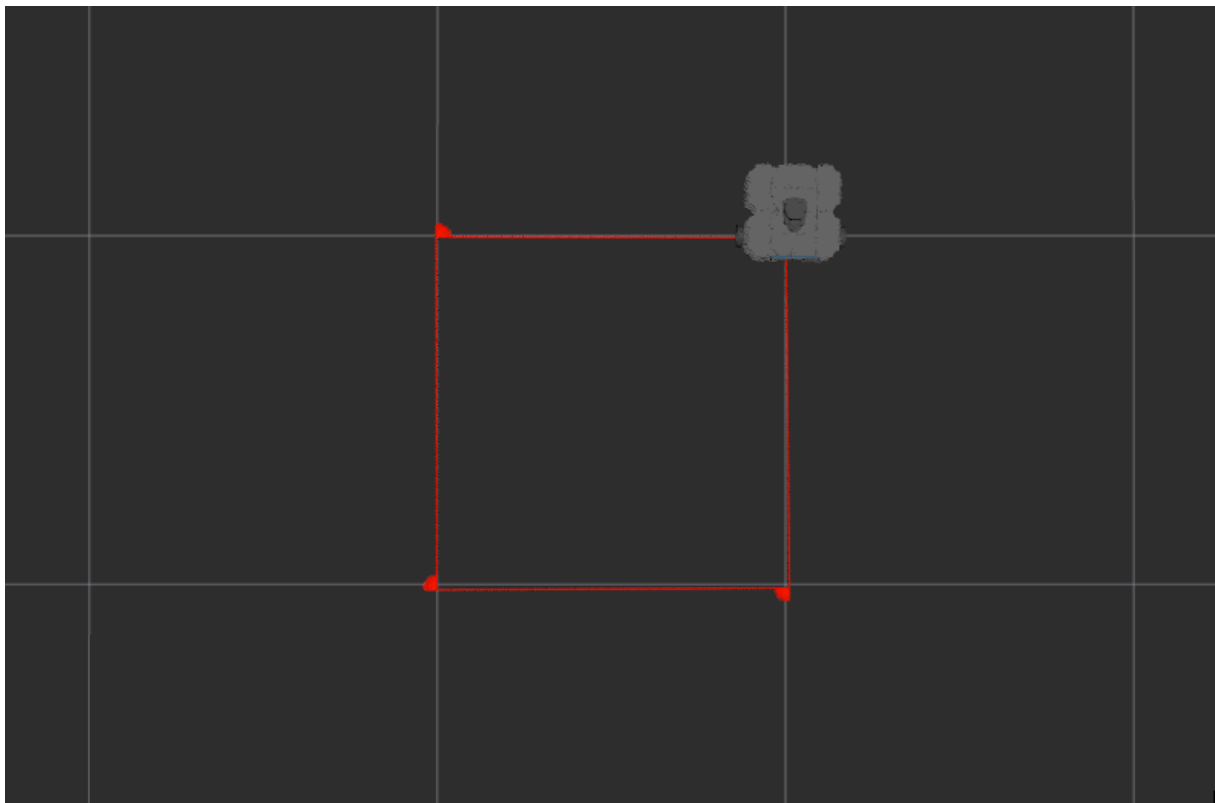
El robot avanza 3 metros y gira  $-120^\circ$  (horario) tres veces. El ángulo exterior de un triángulo equilátero es  $\frac{360^\circ}{3} = 120^\circ$ , con signo negativo para girar a la derecha.



#### MODO 2: CUADRADO DE 1M DE LADO

```
elif movement_type == 2:  
    for _ in range(4):  
        self.steps.append(('forward', 1))  
        self.steps.append(('rotate', -90))
```

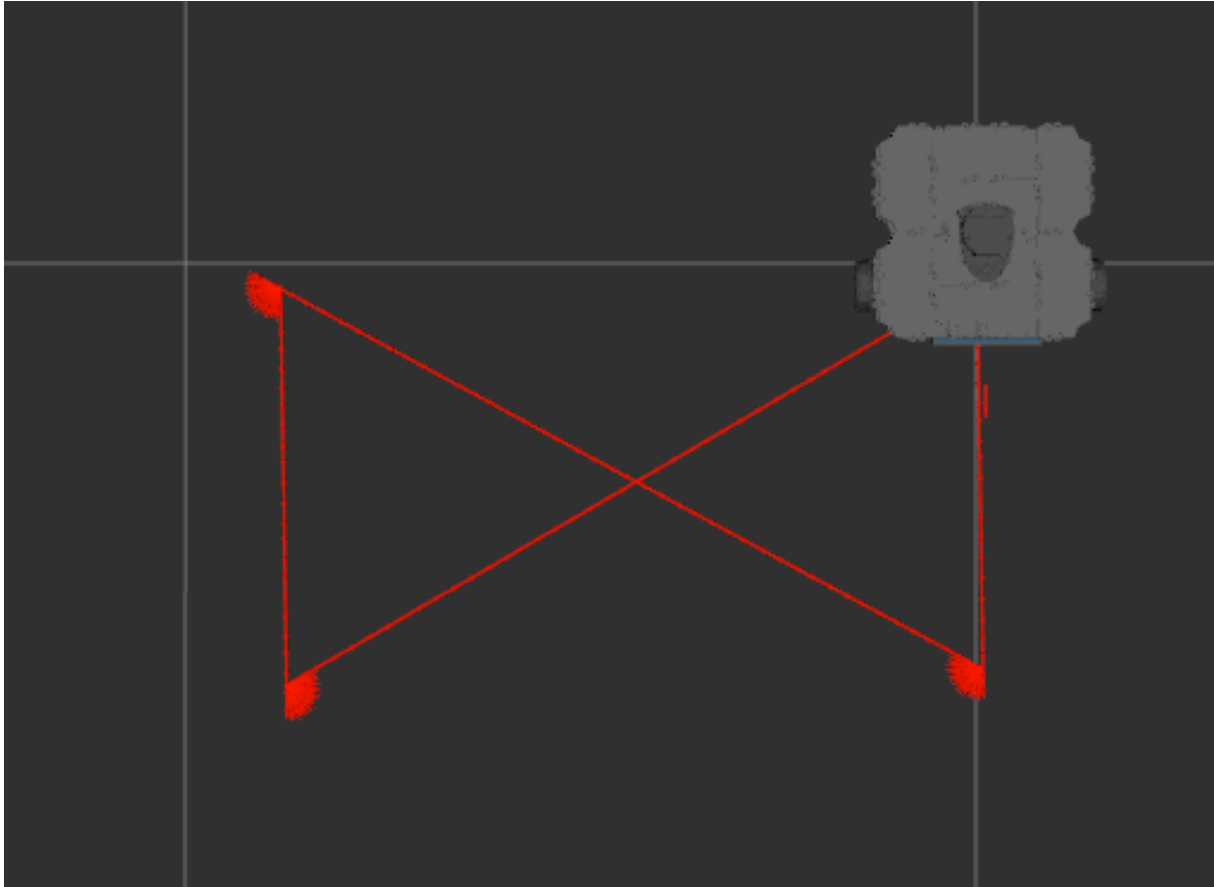
El robot avanza 1 metro y gira  $-90^\circ$  cuatro veces, dibujando un cuadrado.



### MODO 3: FIGURA DE INFINITO (BOWTIE)

```
elif movement_type == 3:
    self.steps.append(('forward', 0.5))
    self.steps.append(('rotate', -120))
    self.steps.append(('forward', 1))
    self.steps.append(('rotate', 120))
    self.steps.append(('forward', 0.5))
    self.steps.append(('rotate', 120))
    self.steps.append(('forward', 1))
    self.steps.append(('rotate', -120))
```

La figura de infinito se construye como dos triángulos simétricos (un *bowtie*). El robot avanza 0.5m, gira para trazar el primer triángulo, vuelve al centro y traza el segundo triángulo en espejo.



## EJECUCIÓN

**Terminal 1:** Lanzar Gazebo con el Turtlebot 3

```
export TURTLEBOT3_MODEL=waffle
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

**Terminal 2:** Lanzar RViz2 para visualizar la trayectoria en tiempo real

```
rviz2
```

Se configura el *display* **Odometry** sobre el topic `/odom` con **Keep** elevado ( $> 1000$ ) para acumular la traza, y se establece el *Fixed Frame* a `odom`. Opcionalmente se añade el *display* **RobotModel** para visualizar el robot.

**Terminal 3:** Ejecutar el movimiento deseado

```
ros2 run p3_pkg movimiento <modo> # 0, 1, 2 o 3
```

## EJERCICIO 2

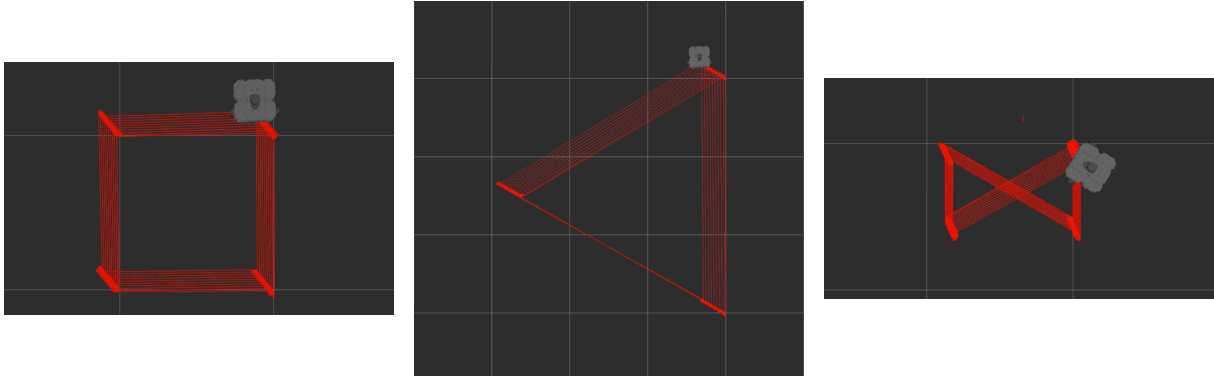
Ejecutar 10 veces el cuadrado, triángulo e infinito y analizar la acumulación de error

---

Se ejecutan los modos 1, 2 y 3 del nodo `movimiento.py` diez veces consecutivas sin resetear la simulación entre ejecuciones. RViz2 acumula la traza de odometría, permitiendo visualizar el *drift* progresivo.

### RESULTADOS

Al repetir cada trayectoria 10 veces, se observa que el robot **no vuelve exactamente a la posición original** en cada iteración. El error se acumula progresivamente, haciendo que la trayectoria se desplace respecto al punto de inicio.



### PREGUNTA

¿El robot acaba en la posición original? ¿A qué puede deberse esto?

---

No, el robot **no vuelve exactamente** a la posición inicial tras cada repetición. El error se acumula con cada iteración, desplazando progresivamente el punto de retorno.

Este problema se conoce como **drift odométrico** y es un problema fundamental de la odometría basada en ruedas (*dead reckoning*). Es la razón por la que los robots reales emplean sensores adicionales (LiDAR, cámaras, IMU) para corregir su estimación de posición.

### EJERCICIO 3

Nodo `aparcamiento.py` para estacionar el Turtlebot 3 en un espacio delimitado

Se ha implementado el nodo `aparcamiento.py` dentro del paquete `p3_pkg`.

#### ENTORNO DE SIMULACIÓN

Para lanzar el entorno de aparcamiento:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch p3_pkg parking_tb3.launch.py
```

El mundo `parking.world` define un espacio de aparcamiento con las siguientes coordenadas:

- › Pared izquierda:  $x \approx 1.0$
- › Pared derecha:  $x \approx 2.0$
- › Pared trasera:  $y \approx -1.95$
- › Entrada abierta:  $y \approx -1.0$
- › Centro del hueco:  $x \approx 1.5$

El robot arranca en  $(0, 0)$  mirando hacia  $+X$ .

#### ESTRATEGIA DE APARCAMIENTO

Se realiza una maniobra de **aparcamiento en marcha atrás** (*reverse bay parking*) en tres pasos:

1. **Avanzar 1.5m:** el robot se desplaza en línea recta hacia  $+X$  hasta alinearse con el centro del hueco ( $x \approx 1.5$ ).
2. **Girar 90°:** rotación antihoraria para orientar el robot hacia  $+Y$ , quedando de espaldas a la entrada del espacio.
3. **Marcha atrás 1.4m:** el robot entra marcha atrás en el hueco hasta quedar centrado.

```
def run(self):
    self.steps.append(('forward', 1.5))
    self.steps.append(('rotate', 90))
    self.steps.append(('reverse', 1.4))
```

#### CÓDIGO

```
class AparcamientoNode(Node):
    def __init__(self):
        super().__init__('aparcamiento_node')
        self.pub = self.create_publisher(Twist, 'cmd_vel', 10)
        self.create_subscription(Odometry, 'odom', self._odom_cb, 10)
        self.steps = deque()
        # ... (estado interno: posición, ángulo, acción actual)

    def _forward_ctrl(self):
        dist = math.hypot(self.x - self.origin_x, self.y - self.origin_y)
        remaining = self.target - dist
        if remaining < LINEAR_TOLERANCE:
            self._stop()
            self._next()
        else:
```

```

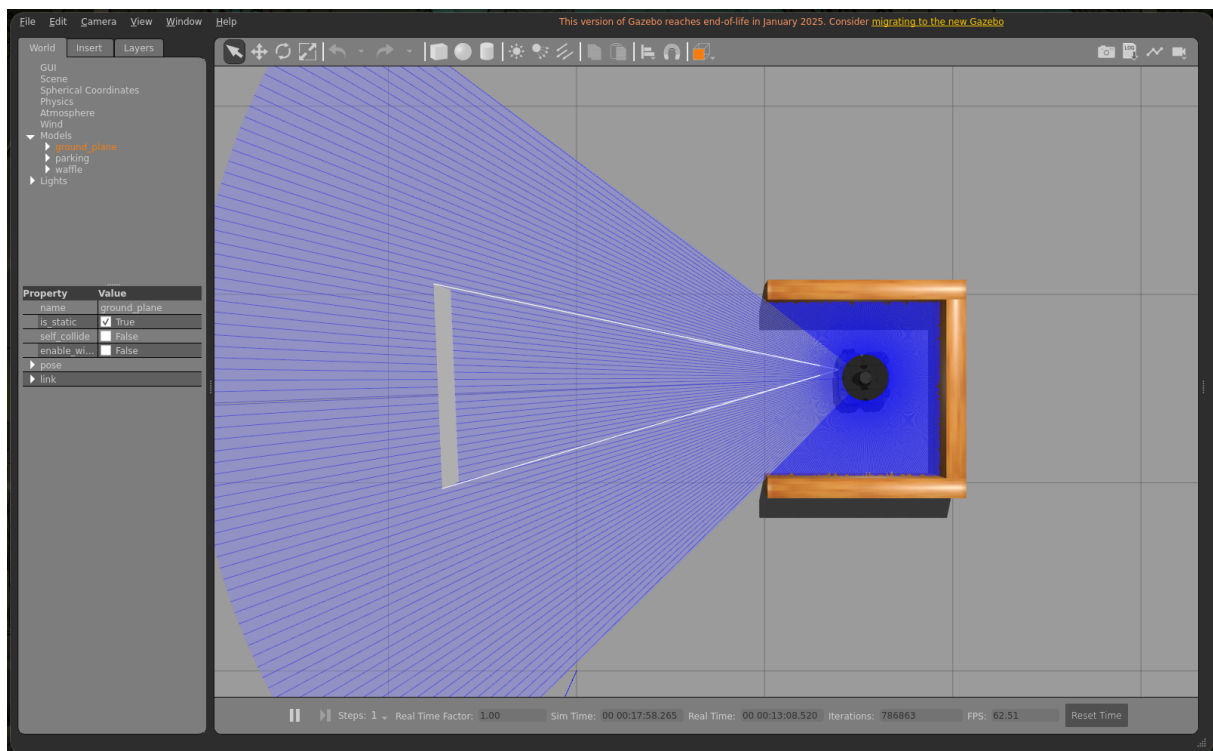
speed = min(SPEED, max(0.05, remaining * 1.5))
if self.action == 'reverse':
    speed = -speed
msg = Twist()
msg.linear.x = speed
self.pub.publish(msg)

def _rotate_ctrl(self):
    angle = math.atan2(
        math.sin(self.theta - self.origin_theta),
        math.cos(self.theta - self.origin_theta))
    remaining = self.target - angle
    if abs(remaining) < ANGULAR_TOLERANCE:
        self._stop()
        self._next()
    else:
        w = min(ANGLE_SPEED, max(0.05, abs(remaining) * 1.5))
        msg = Twist()
        msg.angular.z = w if remaining > 0 else -w
        self.pub.publish(msg)

```

## EJECUCIÓN

```
ros2 run p3_pkg aparcamiento
```



## EJERCICIO 4

### Corrección de errores de odometría mediante un controlador PID

Como se ha observado en los Ejercicios 1 y 2, el robot no ejecuta las trayectorias de forma perfecta. Para corregir estos errores se ha implementado un **controlador PID** (*Proportional-Integral-Derivative*) dentro del nodo `movimiento.py`, activable mediante el flag `--pid`.

```
ros2 run p3_pkg movimiento 2 --pid # cuadrado con PID
ros2 run p3_pkg movimiento 2 10 --pid # cuadrado 10 veces con PID
```

Se implementan dos controladores PID independientes:

**1. PID de rotación:** controla los giros puros (`angular.z`):

```
KP_ROT = 0.5 # Proporcional: corrección suave
KI_ROT = 0.01 # Integral: compensa errores constantes
KD_ROT = 0.05 # Derivativo: amortigua oscilaciones
```

**2. PID de corrección de heading:** mantiene la línea recta durante el avance:

```
KP_HDG = 1.0 # Proporcional: corrección moderada
KI_HDG = 0.0 # Integral: desactivado (no hay sesgo constante)
KD_HDG = 0.1 # Derivativo: suaviza las correcciones
```

Los valores se han ajustado a partir de implementaciones de referencia para el Turtlebot 3.

Al iniciar cada avance, se memoriza el *heading* actual como objetivo (`target_theta`). Si durante el avance el robot se desvía, el PID aplica una corrección en `angular.z` para reconducirlo.

#### IMPLEMENTACIÓN DEL PID

```
def _pid(self, error, kp, ki, kd):
    now = self.get_clock().now().nanoseconds / 1e9
    dt = now - self._prev_time if self._prev_time else 0.0
    self._prev_time = now

    self._integral += error * dt
    self._integral = max(-1.0, min(1.0, self._integral)) # anti-windup

    derivative = (error - self._prev_error) / dt if dt > 0 else 0.0
    self._prev_error = error

    return kp * error + ki * self._integral + kd * derivative
```

El mecanismo de **anti-windup** limita la integral entre  $[-1, 1]$  para evitar que se acumule un valor excesivo cuando el error persiste.

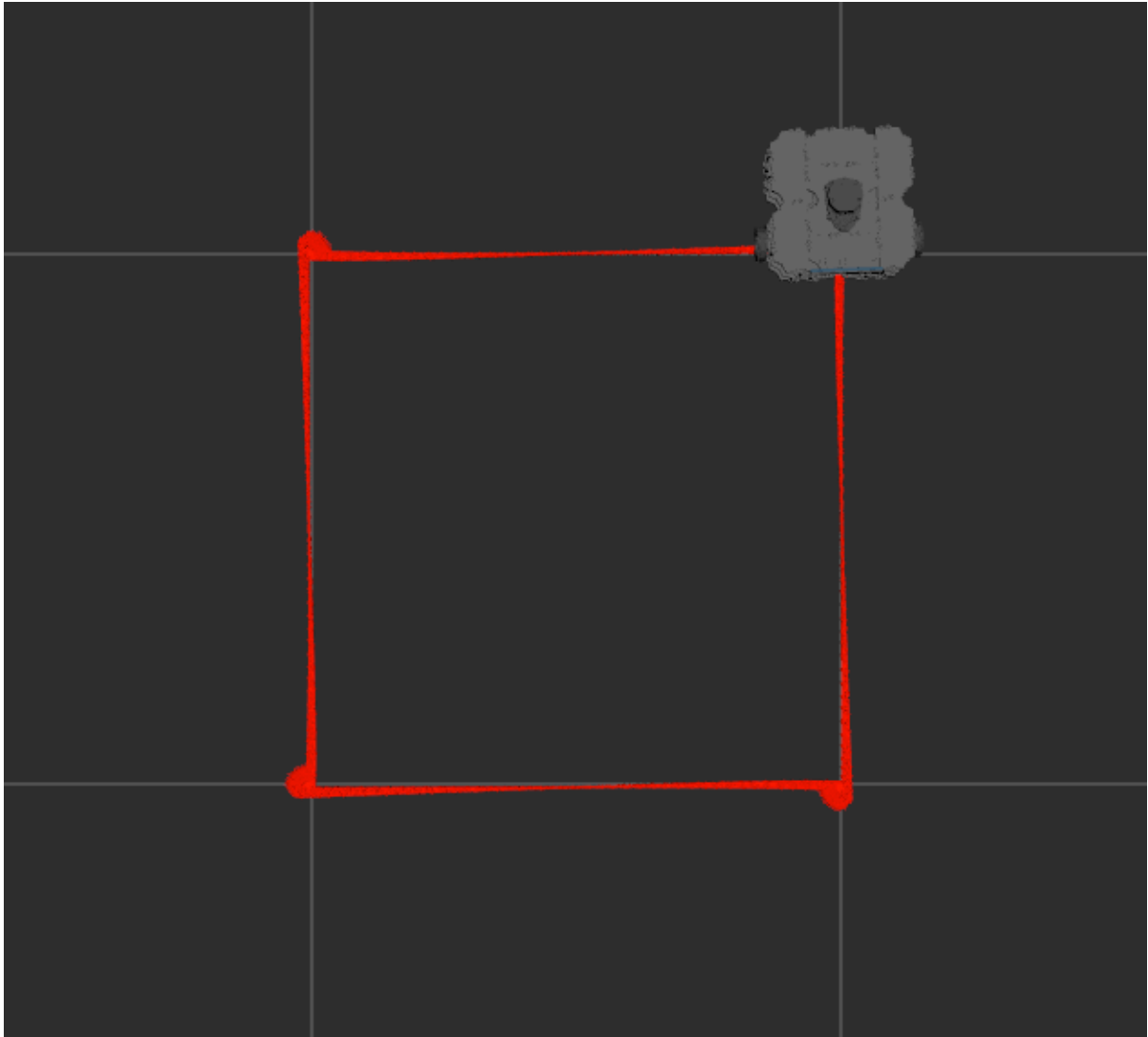
#### TOLERANCIA ANGULAR

Con PID activo, la tolerancia angular se reduce a 0.002 rad ( $\approx 0.11^\circ$ ), frente a los 0.02 rad ( $\approx 1.15^\circ$ ) del modo sin PID. Esto es posible porque el PID desacelera suavemente al aproximarse al ángulo objetivo, mientras que el control a lazo abierto necesita una tolerancia mayor para evitar sobrepasos.

#### RESULTADO

Con el controlador PID, las trayectorias más precisas:

- › Los giros se completan con un error inferior a 0.002 radianes ( $\approx 0.11^\circ$ )
- › Los avances mantienen el *heading* con correcciones en tiempo real
- › Las velocidades se modulan proporcionalmente a la distancia restante, evitando sobrepasos

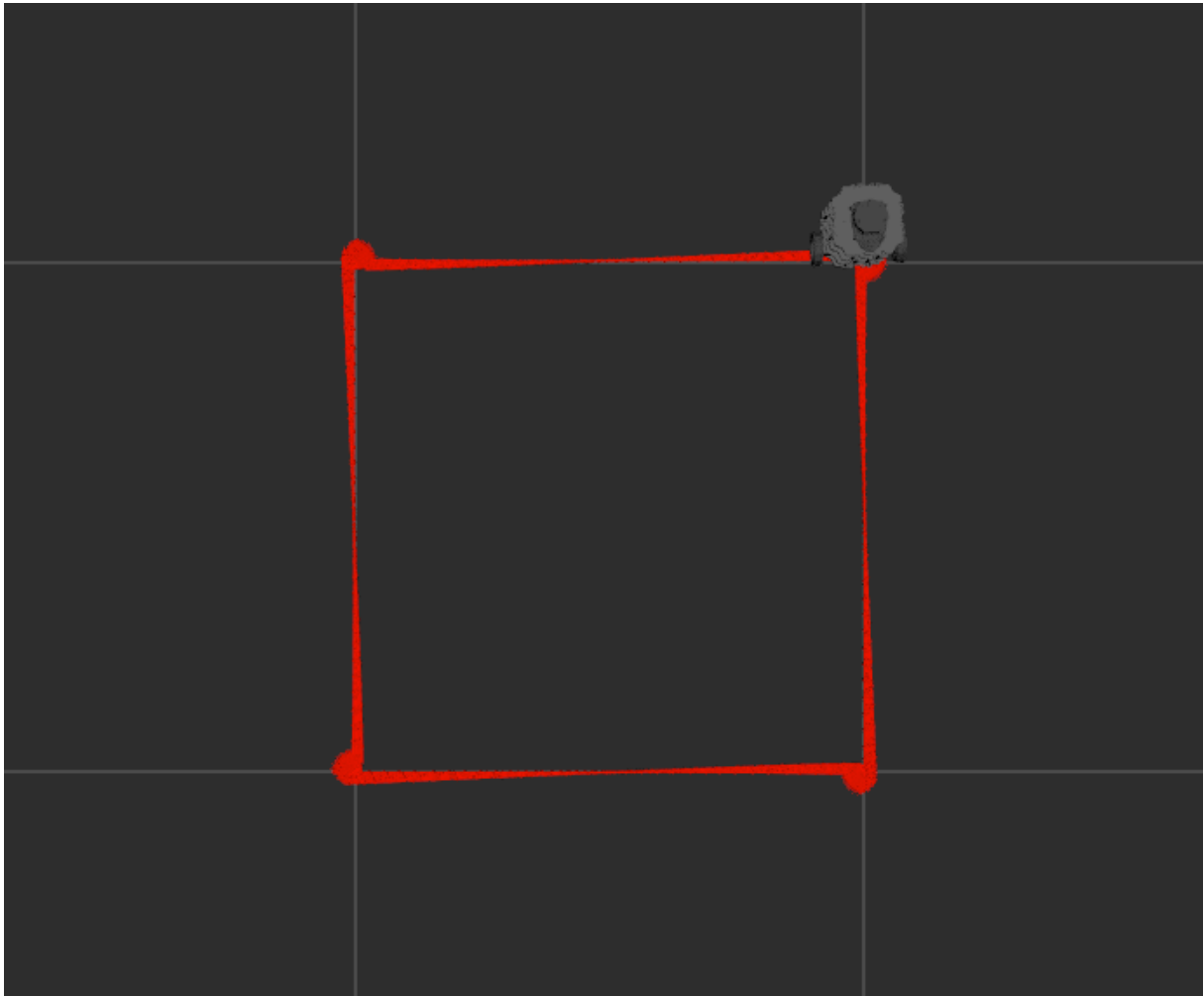


## EJERCICIO 5

Repetir los ejercicios con el Turtlebot 3 Burger y comparar resultados

Se repiten los ejercicios 1-4 empleando el modelo **Burger** en lugar del **Waffle** :

```
export TURTLEBOT3_MODEL=burger
```



### PREGUNTA

¿Observas diferencias en la precisión de los movimientos entre Burger y Waffle? ¿Cuál robot acumula más error? ¿En qué tipo de movimiento se nota más la diferencia?

Sí, hay diferencias aunque no son enormes. Al ejecutar 10 repeticiones del cuadrado con PID activado se obtiene:

| Métrica            | Waffle         | Burger         |
|--------------------|----------------|----------------|
| Posición final     | (−0.02, −0.01) | (−0.02, −0.02) |
| Heading final      | 2.23°          | 2.73°          |
| Drift angular/iter | ~0.22°         | ~0.27°         |

Burger acumula algo más de error, sobre todo angular (un ~23% más de drift por iteración). Donde más se nota es en los **giros**: como tiene las ruedas más juntas (0.160 m vs 0.288 m), cualquier pequeña diferencia de velocidad entre ruedas se traduce en un desvío angular mayor.

#### PREGUNTA

¿Burger realiza giros más amplios o más cerrados que Waffle?

Burger hace giros **más cerrados**. Al tener las ruedas más juntas (0.160 m vs 0.288 m en Waffle), el radio de giro es más pequeño.

#### PREGUNTA

Al implementar controladores para corregir errores de trayectoria: ¿Los mismos parámetros PID funcionan igual para ambos robots?

Sí, los mismos parámetros PID ( `KP_ROT=0.5` , `KI_ROT=0.01` , `KD_ROT=0.05` ) funcionan bien para los dos robots. Ninguno se vuelve inestable ni oscila, y ambos completan las 10 repeticiones del cuadrado sin problemas. Eso sí, Burger acumula algo más de drift angular (~0.27°/iter vs ~0.22°/iter en Waffle), así que el PID no llega a compensar del todo el error extra que introduce su menor separación de ruedas. Para que Burger igualase a Waffle habría que subir un poco las ganancias, sobre todo la proporcional, para que corrija más agresivamente esas pequeñas desviaciones.

#### PREGUNTA

Durante la tarea de aparcamiento: ¿Cuál robot realiza la maniobra con menos correcciones?

Waffle aparca con menos correcciones. Al ser más pesado y tener la base más ancha (0.266 × 0.266 m), sus trayectorias son más estables y no se desvía tanto. Burger es bastante más compacto (0.140 × 0.140 m), así que le sobra espacio dentro del hueco, pero necesita más ajustes correcciones.

#### PREGUNTA

Inspecciona las propiedades URDF de ambos robots: ¿Cómo afectan las diferencias a la precisión y estabilidad?

Las propiedades URDF se encuentran en el paquete `turtlebot3_gazebo/urdf/`. Las principales diferencias son:

| Propiedad            | Burger          | Waffle               |
|----------------------|-----------------|----------------------|
| Radio de rueda       | 0.033 m         | 0.033 m              |
| Separación de ruedas | 0.160 m         | 0.288 m              |
| Masa base            | 0.826 kg        | 1.373 kg             |
| Caja de colisión     | 0.140 × 0.140 m | 0.266 × 0.266 m      |
| Ruedas caster        | 1 trasera       | 2 traseras           |
| Sensor LiDAR         | LDS-01 (360°)   | LDS-01 (360°)        |
| Cámara               | No              | Intel RealSense R200 |

Lo que más afecta es la **separación de ruedas**: Waffle tiene la base casi el doble de ancha (0.288 vs 0.160 m), lo que le da más estabilidad en línea recta. En un robot diferencial, el error angular de la odometría depende de  $\Delta v/L$  (siendo  $L$  la separación entre ruedas), así que con la misma imprecisión en las ruedas, Burger acumula más error angular.

La **masa** también se nota: Waffle pesa más (1.373 kg base vs 0.826 kg), tiene más inercia y eso hace que sus movimientos sean más suaves y predecibles. Burger es más ágil, pero también más sensible a cualquier perturbación.

## Parte 2: Resolución de un Laberinto

En esta segunda parte de la práctica se utiliza el Turtlebot 3 equipado con un sensor LiDAR para resolver un laberinto de manera autónoma. Para ello se crea un nuevo paquete de ROS llamado `maze_pkg`, incluyendo todas las dependencias necesarias. Este paquete contendrá los nodos y scripts requeridos para la navegación del robot dentro del laberinto, empleando técnicas de percepción y control basadas en los datos del LiDAR.

Para comenzar, se lanza en un mundo vacío por un lado el Turtlebot 2 y por otro el Turtlebot 3, y se contesta a las siguientes preguntas para cada robot:

### Preguntas teóricas

#### PREGUNTA 12

¿Cuál es el topic asociado al LiDAR? ¿Cuál es la tipología de los mensajes?

El topic asociado al LiDAR es `/scan` y el tipo de mensaje publicado es `sensor_msgs/msg/LaserScan`. Se puede verificar con:

```
ros2 topic info /scan
```

Lo cual nos devuelve:

```
Type: sensor_msgs/msg/LaserScan
Publisher count: 1
Subscription count: 0
```

Si mostramos información sobre el tipo de mensaje con el siguiente comando:

```
ros2 interface show sensor_msgs/msg/LaserScan
```

Obtendremos todos los campos que componen el mensaje:

```
# Single scan from a planar laser range-finder
#
# If you have another ranging device with different behavior (e.g. a sonar
# array), please find or create a different message, since applications
# will make fairly laser-specific assumptions about this data

std_msgs/Header header
  builtin_interfaces/Time stamp
  int32 sec
  uint32 nanosec
  string frame_id
float32 angle_min
float32 angle_max
float32 angle_increment
float32 time_increment
float32 scan_time
float32 range_min
float32 range_max
```

```
float32[] ranges
float32[] intensities
```

### PREGUNTA 13

¿Cuál es el rango de distancias que puede medir el LiDAR? ¿Cuál es el rango angular de escaneo que tiene el LiDAR? ¿Cuál es el origen de referencia del LiDAR?

Para poder comprobar los rangos de distancia y ángulo del LiDAR, así como su origen de referencia, se puede «escuchar» uno de los mensajes publicados en el topic `/scan` con el siguiente comando:

```
ros2 topic echo /scan --once
```

Entre la información publicada en el mensaje, se pueden encontrar los siguientes campos relevantes:

- > range\_min: 0.11999999731779099
- > range\_max: 3.5
- > angle\_min: 0.0
- > angle\_max: 6.28000020980835
- > frame\_id: «base\_scan»

Con esta información se puede concluir que el LIDAR puede medir distancias desde aproximadamente 0.12 metros hasta 3.5 metros, con un rango angular completo de  $360^\circ$  (de 0 a  $2\pi$  radianes). El origen de referencia del LiDAR es el frame `base_scan`, lo que nos indica que el centro de coordenada de la información devuelta por el LIDAR es la propia posición del sensor.

## Ejercicios

### EJERCICIO 1

Nodo `res_maze.py` para resolver el laberinto con el Turtlebot 3

Se ha implementado el nodo para resolver el primer laberinto (`maze_1.world`) utilizando la información del LIDAR para detectar las paredes del laberinto. Para ellos hemos usado el algoritmo de seguimiento de pared derecha, que consiste en mantener la pared a la derecha del robot mientras se avanza. El robot gira a la derecha si no hay pared a su derecha, avanza recto mientras haya pared a su derecha, y gira derecha o izquierda si detecta un obstáculo frontal.

Para resolverlo se han definido los cuatro estados siguientes:

1. **Avanzar recto:** El robot avanza manteniendo la pared a su derecha, corrigiendo su trayectoria si se aleja demasiado de ella.
2. **Girar a la izquierda:** El robot gira a la izquierda para evitar un obstáculo frontal, hasta que la vía quede despejada.
3. **Girar a la derecha para evitar un obstáculo:** El robot gira a la derecha en caso de encontrar un obstáculo frontal.
4. **Girar a la derecha y avanzar:** El robot gira a la derecha mientras avanza, hasta tener una pared a su derecha.

Además, se ha definido un estado extra a los cuatro típicos, para conseguir que el robot comience, debido a que en la posición inicial no se encuentra ninguna pared por lo que no se cumplía ninguna de las condiciones anteriores. En este estado inicial, el robot avanza recto hasta encontrar una pared, momento en el que comienza a aplicar el algoritmo de seguimiento de pared derecha.

Para lograr obtener la información del LiDAR se ha creado un callback que se suscribe al topic correspondiente, y que procesa los datos de escaneo para determinar la presencia de paredes en las direcciones frontal y derecha. En función de esta información, el nodo decide qué acción tomar en cada momento. Para detectar un muro, se usa un ángulo de «visión» y se calcula la distancia mínima a un obstáculo dentro de ese rango angular, durante las pruebas también se probó a usar la media de las distancias dentro del rango, pero se observó que el mínimo era más efectivo para evitar colisiones.

También se han definido unos umbrales de distancia para considerar que hay una pared, quedando establecidos finalmente en:

```
UMBRAL_FRENTE = 1.2 # Distancia mínima para considerar que hay una pared al frente (m)
UMBRAL_DERECHA = 1.2 # Distancia mínima para considerar que hay una pared a la derecha (m)
```

Como resultado, el robot es capaz de resolver el laberinto siguiendo la pared derecha, se puede comprobar en el siguiente [video de la ejecución](#) del nodo o lanzando el entorno en Gazebo y ejecutando el nodo:

› Primera terminal: Lanzar el entorno de Gazebo con el laberinto

```
ros2 launch maze_pkg maze_1.launch.py
```

› Segunda terminal: Ejecutar el nodo de resolución del laberinto

```
ros2 run maze_pkg maze_solver_node
```

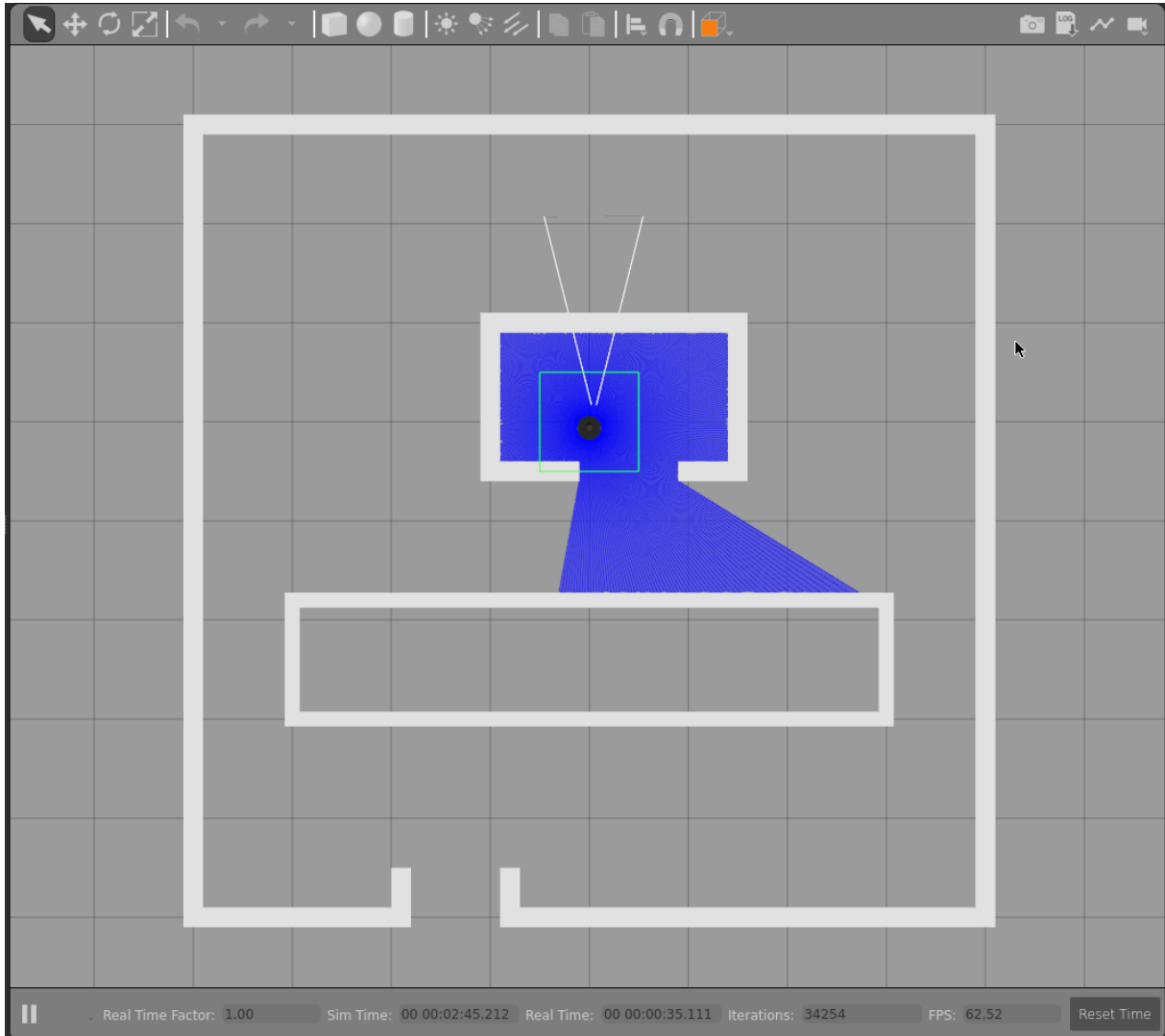
## EJERCICIO 2

Ejecutar el algoritmo en el segundo laberinto ( `maze_2.world` ) y analizar su comportamiento

En la carpeta `worlds` existe un segundo mapa con un modelo de laberinto distinto llamado `maze_2.world` . Se lanza este nuevo entorno en Gazebo y se ejecuta el algoritmo de resolución del Ejercicio 1:

```
ros2 launch maze_pkg maze_2.launch.py
```

### RESULTADO



### PREGUNTA 14

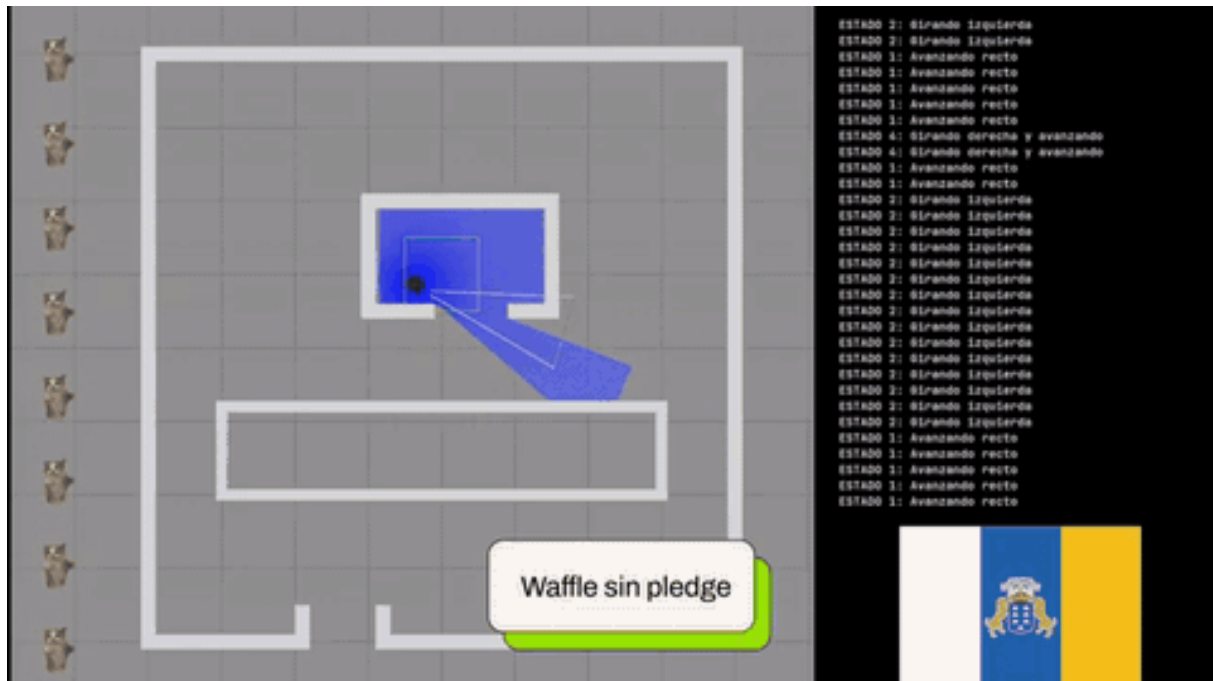
¿Qué problemática observas en este tipo de escenarios?

El problema principal de `maze_2` es que hay paredes internas que no están conectadas con las paredes exteriores del laberinto (las que tienen la salida). Esto hace que el algoritmo de seguimiento se quede dando vueltas indefinidamente alrededor de una «isla» interior sin llegar nunca a la salida, ya que la pared que sigue no lleva a ningún lado.

### PREGUNTA 15

¿Es el robot capaz de resolver este laberinto? Si no es así, justifica tu respuesta. ¿Qué información crees que necesita el robot para poder llegar a resolverlo?

Con un simple wall-following, no. Si el robot empieza siguiendo una pared que forma parte de una isla desconectada, se queda en bucle sin alcanzar la salida. Para resolverlo necesitaría información adicional, como un mapa del laberinto (por ejemplo mediante SLAM) o algún mecanismo de memoria que le permita detectar que está repitiendo el mismo recorrido y cambiar de estrategia.



Para mejorar el comportamiento ante ese bucle, se ha combinado el seguimiento de pared derecha con el algoritmo de **Pledge**: se integra la rotación neta del robot (mediante odometría) y, cuando el contador de giros vuelve a cero en un punto donde puede avanzar hacia el exterior de la «isla», se abandona el **wall following** y se busca de nuevo una pared a seguir. La lógica actual está en [ros2\\_ws/src/maze\\_pkg/maze\\_pkg/res\\_maze.py](#) (nodo [maze\\_solver](#)).

#### ACUMULACIÓN DE ROTACIÓN ('/ODOM')

El callback de odometría obtiene el *yaw* a partir del cuaternión, calcula el incremento angular entre lecturas corrigiendo el salto  $\pm 180^\circ$ , y acumula el giro en [rotacion\\_acumulada](#), que es la magnitud que usa Pledge para saber si el robot ha completado un ciclo de vueltas respecto a la orientación de referencia:

```
if self.angulo_previo is None:
    self.angulo_previo = grados
    return

delta = grados - self.angulo_previo
if delta > 180.0:
    delta -= 360.0
elif delta < -180.0:
    delta += 360.0

self.rotacion_acumulada += delta
self.angulo_previo = grados
```

#### MÁQUINA DE ESTADOS PLEDGE + PARED DERECHA

`estado_pledge == 0` avanza en línea recta hasta detectar pared a la derecha; entonces pasa a `estado_pledge == 1` y aplica la misma lógica de cuatro casos que el seguimiento de pared (avanzar, girar izquierda, girar derecha, o combinar giro y avance). La novedad es la condición de escape: en estado 1, si la rotación acumulada está cerca de cero y no hay obstáculo frontal, se vuelve al estado 0 y se intenta «salir» recto, rompiendo el circuito alrededor de la isla:

```
if self.estado_pledge == 0:
    if self.pared_derecha:
        self.estado_pledge = 1
        self.rotacion_acumulada = 0.0
    else:
        cmd.linear.x = VEL_LINEAL
        cmd.angular.z = 0.0
elif self.estado_pledge == 1:
    if abs(self.rotacion_acumulada) < TOLERANCIA_GRADOS and not self.pared_frente:
        self.estado_pledge = 0
        self.rotacion_acumulada = 0.0
        cmd.linear.x = VEL_LINEAL
        cmd.angular.z = 0.0
    else:
        if self.pared_derecha and not self.pared_frente:
            cmd.linear.x = VEL_LINEAL
            cmd.angular.z = 0.0
        elif self.pared_derecha and self.pared_frente:
            cmd.linear.x = 0.0
            cmd.angular.z = VEL_ANGULAR
        elif self.pared_frente and not self.pared_derecha:
            cmd.linear.x = 0.0
            cmd.angular.z = -VEL_ANGULAR
        else:
            cmd.linear.x = VEL_LINEAL / 4.0
            cmd.angular.z = -VEL_ANGULAR
```

Con esta capa adicional, el Turtlebot 3 Waffle puede completar el laberinto `maze_2` en la simulación; a continuación se muestra una captura animada del comportamiento con Pledge activo:



### EJERCICIO 3

#### Repetir los Ejercicios 1 y 2 con el Turtlebot 3 Burger

---

Se repiten los ejercicios anteriores empleando el modelo **Burger** :

```
export TURTLEBOT3_MODEL=burger
```

### PREGUNTA 16

¿Qué diferencias observas respecto al otro modelo? Detalla claramente las diferencias que observes.

---

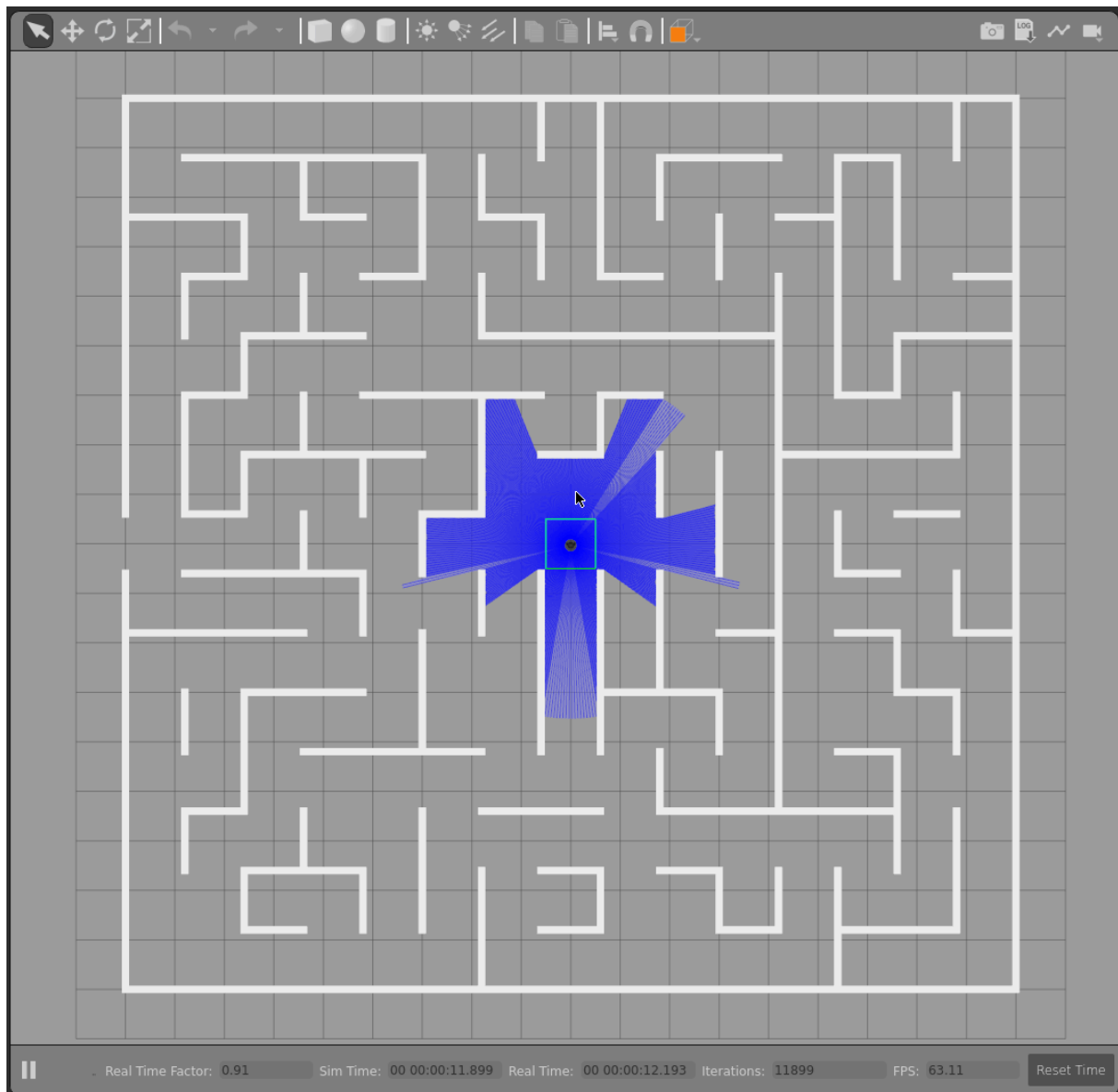
#### EJERCICIO 4

### Generación de un laberinto personalizado y resolución con ambos modelos

Se genera un nuevo laberinto personalizado y se prueban ambos modelos del Turtlebot 3 para resolverlo. Para la creación del mapa se investiga cómo generar modelos y mundos en Gazebo. Como guía para el diseño del laberinto se puede utilizar el siguiente generador:

[Maze Generator](#)

#### DISEÑO DEL LABERINTO



[Resultado con Burger](#)